52. Design Twitter / X

Problem Statement

Design a social media platform like Twitter/X that allows users to post short messages (tweets), follow other users, view a personalized home timeline (news feed), search content, and engage with tweets via likes, retweets, and replies. The system must handle hundreds of millions of users, celebrity accounts with 100M+ followers, and viral content that can spike traffic 10-100x in seconds.

Clarifying Questions to Ask

1. **Core features**: Tweet, follow, timeline, search, notifications — which to prioritize?
2. **Tweet length**: 280 characters? Or also support long-form posts, images, video?
3. **Timeline**: Chronological, algorithmic (ranked), or both?
4. **Celebrity problem**: How to handle users with 100M+ followers? (fan-out challenge)
5. **Real-time**: How fresh should the timeline be? (seconds vs minutes)
6. **Search**: Full-text search over all tweets? Trending topics?
7. **Scale**: How many DAU? (300M+ DAU, 500M+ tweets/day)
8. **Media**: Support images, video, polls, or text-only for now?
9. **Notifications**: Real-time push for mentions, likes, retweets?
10. **Ads**: Inject promoted tweets into timeline?

Requirements & Sizing

Functional

- **Tweet**: Post a tweet (text, media, polls), delete own tweets
- **Follow/Unfollow**: Follow other users, see follower/following lists
- **Home Timeline**: Personalized feed of tweets from followed users
- **User Timeline**: All tweets by a specific user (profile page)
- **Like / Retweet / Reply**: Engage with tweets
- **Search**: Full-text search across tweets, users, hashtags
- **Trending**: Real-time trending topics by region

- **Notifications**: Mentions, likes, retweets, new followers

Non-functional

Metric	Target
Timeline latency	< **200ms** p99 for home timeline load
Tweet publish	< **500ms** until visible to author
Tweet fan-out	< **5s** for tweet to reach followers' timelines
Availability	**99.99%** (four 9s)
Scale	**300M DAU**, **500M tweets/day**
Search latency	< **300ms** for keyword search
Storage	Petabytes of tweets + media

Capacity Estimates

```
Users:
- 1B total accounts, 300M DAU
- Average user follows ~200 accounts
- Average user reads timeline ~10 times/day

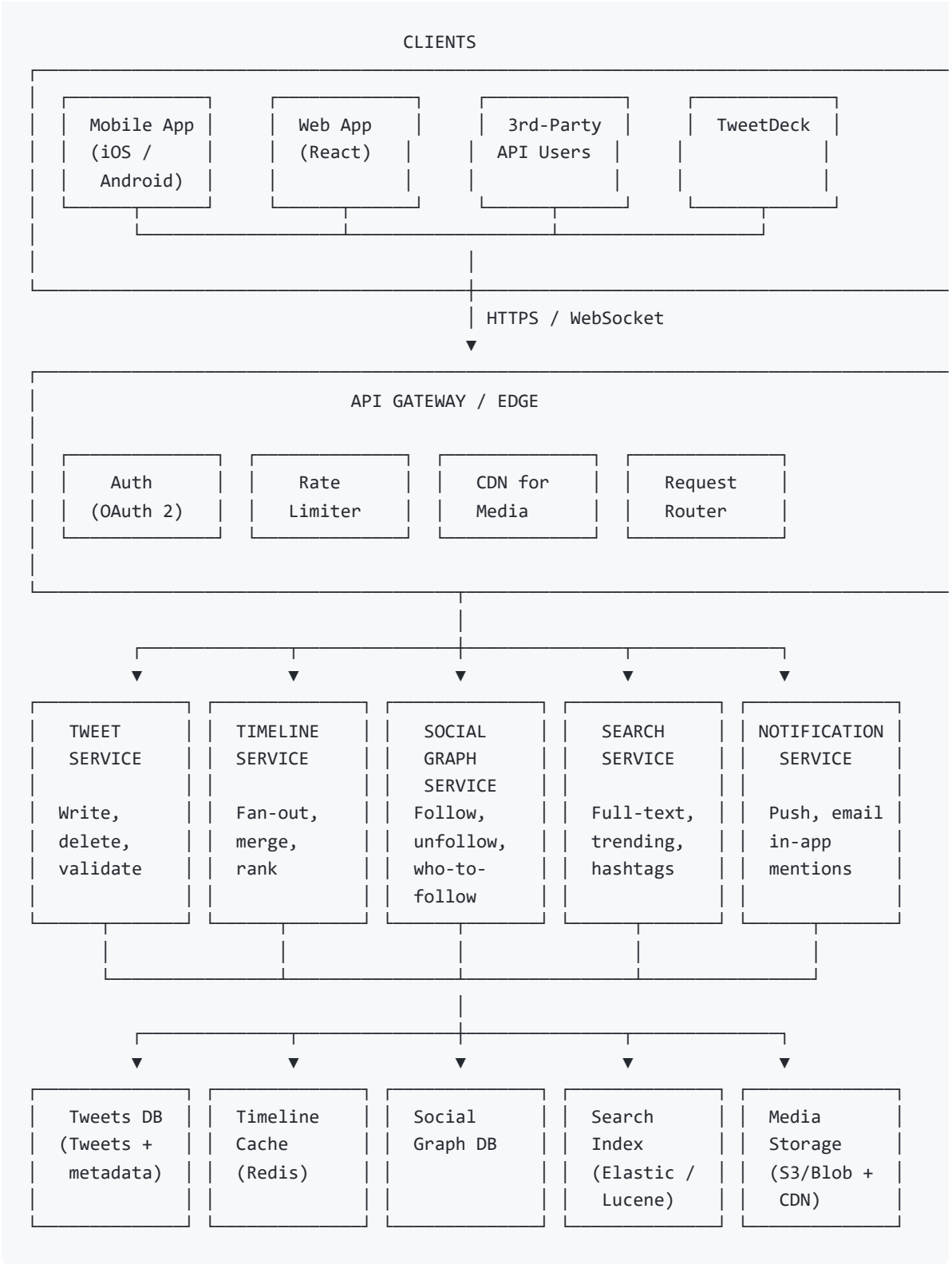
Tweets:
- 500M tweets/day ≈ 6,000 tweets/second (average)
- Peak: 30,000 tweets/second (during major events)
- Average tweet: 200 bytes text + 500 bytes metadata = ~700 bytes
- Daily tweet storage: 500M × 700B ≈ 350 GB/day ≈ 125 TB/year

Timeline reads:
- 300M DAU × 10 reads/day = 3B timeline reads/day
- ≈ 35,000 reads/second (average), 150K+ at peak

Fan-out:
- Average user: 200 followers → 500M × 200 = 100B fan-out ops/day
- Celebrity (100M followers): 1 tweet = 100M writes
- This is THE hard problem
```

High-Level Design

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          TWITTER / X ARCHITECTURE                            │
└──────────────────────────────────────────────────────────────────────────────┘
```



API Design

Tweet APIs

Method	Endpoint	Description
`POST`	`/v2/tweets`	Create a tweet
`DELETE`	`/v2/tweets/{id}`	Delete a tweet
`GET`	`/v2/tweets/{id}`	Get a single tweet
`POST`	`/v2/tweets/{id}/retweet`	Retweet
`POST`	`/v2/tweets/{id}/like`	Like a tweet
`DELETE`	`/v2/tweets/{id}/like`	Unlike a tweet

Timeline APIs

Method	Endpoint	Description
`GET`	`/v2/timeline/home`	Get home timeline (paginated)
`GET`	`/v2/timeline/user/{userId}`	Get user's tweets (profile)
`GET`	`/v2/timeline/home?cursor={cursor}`	Pagination via cursor

Social Graph APIs

Method	Endpoint	Description
`POST`	`/v2/users/{id}/follow`	Follow a user
`DELETE`	`/v2/users/{id}/follow`	Unfollow a user
`GET`	`/v2/users/{id}/followers`	List followers (paginated)
`GET`	`/v2/users/{id}/following`	List following (paginated)

Search APIs

Method	Endpoint	Description
`GET`	`/v2/search?q={query}`	Search tweets
`GET`	`/v2/trends/{region}`	Get trending topics

Create Tweet Request

```
POST /v2/tweets
{
  "text": "System design is fun! #coding",
  "media_ids": ["media_abc123"],
  "reply_to": null,
  "quote_tweet_id": null,
  "poll": null
}
```

Create Tweet Response

```
{
  "id": "1234567890",
  "text": "System design is fun! #coding",
  "author_id": "user_42",
  "created_at": "2026-02-22T19:00:00Z",
  "metrics": {
    "likes": 0,
    "retweets": 0,
    "replies": 0,
    "views": 0
  },
  "entities": {
    "hashtags": [{ "tag": "coding", "start": 26, "end": 33 }]
  }
}
```

Home Timeline Response

```
GET /v2/timeline/home?cursor=cursor_xyz&count=20

{
  "tweets": [
    {
      "id": "1234567890",
      "text": "System design is fun! #coding",
      "author": { "id": "user_42", "name": "Jane", "handle": "@jane" },
      "created_at": "2026-02-22T19:00:00Z",
      "metrics": { "likes": 142, "retweets": 23, "replies": 5, "views": 8420 },
      "ranked_score": 0.87
    }
  ],
  "next_cursor": "cursor_abc",
  "has_more": true
}
```

Tweet Storage

Tweet Table Schema

```
┌──────────────────────────────────────────────────────────────────┐
│                    TWEET STORAGE                                 │
│                                                                  │
│  PRIMARY STORE: Distributed SQL (Vitess / CockroachDB)           │
│                                                                  │
│  tweets table:                                                   │
│  ├── tweet_id       BIGINT (Snowflake ID, time-sortable)         │
│  ├── author_id      BIGINT                                       │
│  ├── text           VARCHAR(280)                                 │
│  ├── media_urls     JSON                                         │
│  ├── reply_to_id    BIGINT (nullable)                            │
│  ├── quote_tweet_id BIGINT (nullable)                            │
│  ├── created_at     TIMESTAMP                                    │
│  ├── is_deleted     BOOLEAN                                      │
│  └── hashtags       TEXT[] (extracted at write time)             │
│                                                                  │
│  Sharding key: tweet_id (Snowflake = time-ordered)               │
│  Why: Most queries are by tweet_id or time range                 │
│                                                                  │
│  tweet_counts table (separate for high-write):                   │
│  ├── tweet_id       BIGINT                                       │
│  ├── like_count     INT                                          │
│  ├── retweet_count  INT                                          │
│  ├── reply_count    INT                                          │
│  └── view_count     BIGINT                                       │
│                                                                  │
│  WHY SEPARATE COUNTS?                                            │
│  - Likes/views update millions of times per second               │
│  - Core tweet data is write-once, read-many                      │
│  - Separate table avoids row-lock contention                     │
│  - Counts can be eventually consistent (it's OK to show          │
│    "142 likes" when the true count is 143)                       │
│                                                                  │
│  STORAGE ESTIMATE:                                               │
│  - 500M tweets/day × 700 bytes ≈ 350 GB/day                      │
│  - 5 years ≈ 640 TB of tweet data                                │
│  - Sharded across 100+ database nodes                            │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Snowflake ID (Tweet ID Generation)

```
┌──────────────────────────────────────────────────────────────────┐
│                    SNOWFLAKE ID                                  │
│                                                                  │
```

```
┌──────────────────────────────────────────────────────────────────┐
│  64-bit unique ID, time-sortable:                                │
│                                                                  │
│  ┌──────────┬──────────┬──────────┬──────────┐                   │
│  │  1 bit   │ 41 bits  │  10 bits │  12 bits │                   │
│  │ (unused) │ (millis  │ (machine │ (sequence│                   │
│  │          │  since   │   ID)    │  number) │                   │
│  │          │  epoch)  │          │          │                   │
│  └──────────┴──────────┴──────────┴──────────┘                   │
│                                                                  │
│  Properties:                                                     │
│  ├── Globally unique without coordination                        │
│  ├── Time-sortable (newer tweets = higher IDs)                   │
│  ├── 41 bits = ~69 years of milliseconds                         │
│  ├── 10 bits = 1,024 machines                                    │
│  ├── 12 bits = 4,096 IDs per machine per millisecond             │
│  └── Total: ~4M IDs/second/machine                               │
│                                                                  │
│  Used as: tweet_id, cursor for pagination ("tweets after X")     │
└──────────────────────────────────────────────────────────────────┘
```

The Fan-Out Problem (Core Challenge)

This is the **defining problem** of Twitter's architecture.

Fan-Out on Write (Push Model)

```
┌──────────────────────────────────────────────────────────────────┐
│                    FAN-OUT ON WRITE                              │
│                                                                  │
│  When user A posts a tweet:                                      │
│  1. Write tweet to tweets table                                  │
│  2. Look up A's followers: [B, C, D, ... N]                      │
│  3. For EACH follower, append tweet_id to their timeline cache   │
│                                                                  │
│  User A (1,000 followers) tweets "Hello world"                   │
│       │                                                          │
│       ▼                                                          │
│  Tweet Service → Write to tweets DB                              │
│       │                                                          │
│       ▼                                                          │
│  Fan-out Service → Get followers of A = [B, C, D, ...]           │
│       │                                                          │
│       ▼                                                          │
│  For each follower, append to their timeline cache:              │
│  ┌──────────────────────────────────────┐                        │
│  │  Redis timeline cache (sorted set):  │                        │
│  │  user:B:timeline → [(tweet_id, score=timestamp), ...]         │
│  │  user:C:timeline → [(tweet_id, score=timestamp), ...]         │
│  │  user:D:timeline → [(tweet_id, score=timestamp), ...]         │
```

```
│  └──────────────────────────────────────────┘                    │
│                                                                  │
│  Timeline read = just fetch from Redis (O(1), < 10ms)            │
│                                                                  │
│  ✅ PROS:                                                        │
│  ├── Timeline reads are extremely fast (pre-computed in cache)   │
│  ├── Simple read path: GET user:X:timeline from Redis            │
│  └── Scales reads infinitely (most operations are reads)         │
│                                                                  │
│  ❌ CONS:                                                        │
│  ├── Write amplification: 1 tweet × N followers = N writes       │
│  ├── Celebrity problem: 1 tweet × 100M followers = 100M writes!  │
│  ├── High Redis memory: 300M users × 800 tweet IDs × 8 bytes     │
│  │   ≈ ~2 TB of just timeline cache                              │
│  └── Fan-out delay: 100M writes takes minutes even at 1M/s       │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Fan-Out on Read (Pull Model)

```
┌──────────────────────────────────────────────────────────────────┐
│                    FAN-OUT ON READ                               │
│                                                                  │
│  When user B reads their timeline:                               │
│  1. Look up who B follows: [A, X, Y, Z, ...]                     │
│  2. Fetch recent tweets from each followed user                  │
│  3. Merge and sort by time (or ranking score)                    │
│  4. Return top N                                                 │
│                                                                  │
│  User B opens app → GET /timeline/home                           │
│       │                                                          │
│       ▼                                                          │
│  Timeline Service → Get B's following list = [A, X, Y, Z, ...]   │
│       │                                                          │
│       ▼                                                          │
│  Fetch recent tweets from each (parallel):                       │
│  ├── tweets WHERE author_id = A ORDER BY created_at DESC LIMIT 20│
│  ├── tweets WHERE author_id = X ORDER BY created_at DESC LIMIT 20│
│  ├── tweets WHERE author_id = Y ORDER BY created_at DESC LIMIT 20│
│  └── ... (for all 200 followed users)                            │
│       │                                                          │
│       ▼                                                          │
│  Merge sort → Rank → Return top 20                               │
│                                                                  │
│  ✅ PROS:                                                        │
│  ├── No write amplification (tweet write = just 1 DB write)      │
│  ├── No celebrity problem (no fan-out at write time)             │
│  └── Less memory (no pre-computed timeline caches)               │
│                                                                  │
│  ❌ CONS:                                                        │
│  ├── Slow reads: 200 DB queries merged in real-time              │
│  ├── High read latency: N parallel queries + merge = slow        │
│  └── Doesn't scale: 35K timeline reads/sec × 200 queries each    │
│      = 7M DB queries/sec just for timelines                      │
```

```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Hybrid Approach (Twitter's Actual Solution) ⭐

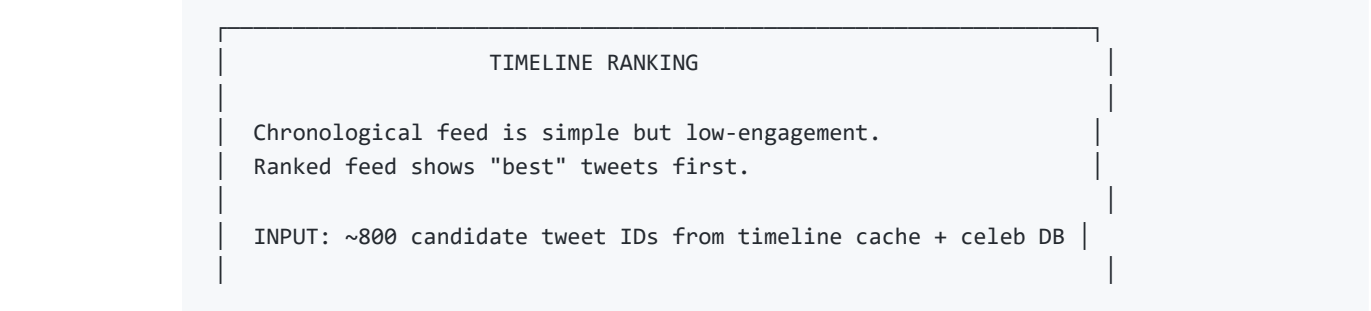
```
┌──────────────────────────────────────────────────────────────────┐
│                    HYBRID FAN-OUT                                │
│                                                                  │
│  RULE: Use fan-out on write for normal users,                    │
│        fan-out on read for celebrities (>500K followers)         │
│                                                                  │
│  When NORMAL user tweets (< 500K followers):                     │
│  └── Fan-out on write → push to all followers' timeline caches   │
│                                                                  │
│  When CELEBRITY tweets (> 500K followers):                       │
│  └── DON'T fan out → just write to tweets DB                     │
│                                                                  │
│  When user reads timeline:                                       │
│  ┌──────────────────────────────────────────────────┐            │
│  │  1. Fetch pre-computed timeline from Redis       │            │
│  │     (contains tweets from normal followed users) │            │
│  │                                                  │            │
│  │  2. For each celebrity the user follows:         │            │
│  │     Fetch their recent tweets from DB (parallel) │            │
│  │     (typically user follows < 20 celebrities)    │            │
│  │                                                  │            │
│  │  3. MERGE both sets, sort/rank, return top N     │            │
│  └──────────────────────────────────────────────────┘            │
│                                                                  │
│  WHY THIS WORKS:                                                 │
│  ├── Normal users (99.9%): Fan-out on write is fine (< 500K)     │
│  ├── Celebrities (0.1%): Skip fan-out, query at read time        │
│  ├── Read merge: User follows maybe 5-20 celebrities             │
│  │   → only 5-20 extra DB queries per timeline read              │
│  ├── Total read cost: Redis GET + 5-20 small DB queries          │
│  │   → still < 50ms                                              │
│  └── Eliminates the 100M-write-per-celebrity-tweet problem       │
│                                                                  │
│  CELEBRITY THRESHOLD:                                            │
│  ├── Configurable (Twitter used ~30K-500K depending on era)      │
│  ├── Can also be dynamic based on engagement rate                │
│  └── Some accounts are "super-celebrities" (>50M) — always pull  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Fan-Out Service Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                    FAN-OUT SERVICE                               │
│                                                                  │
│  Tweet Published                                                 │
```

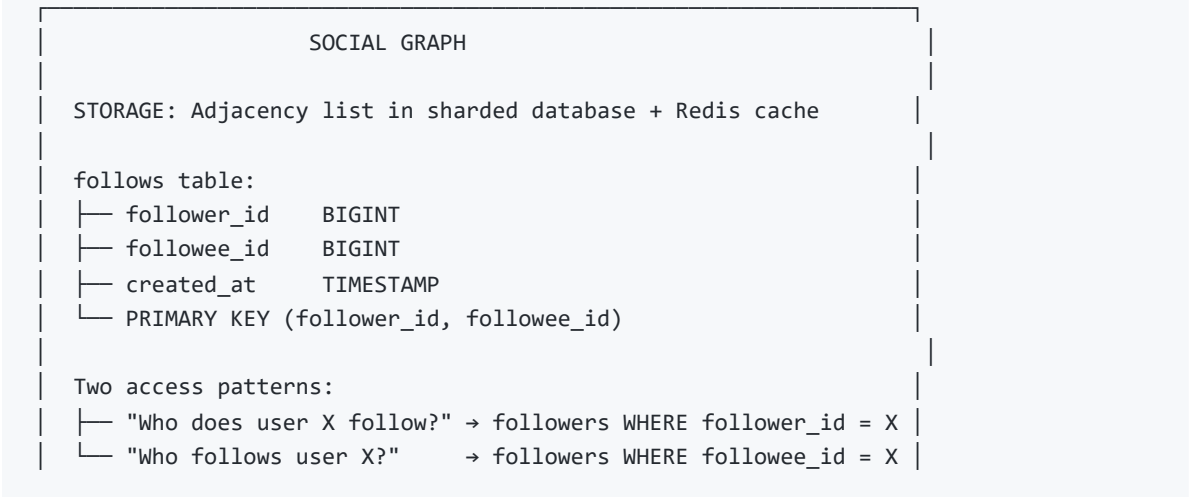


Timeline Ranking (Algorithmic Feed)





Social Graph Service



```
│                                                                  │
│  Sharding: Shard by follower_id (optimized for "following" query)│
│  Secondary index on followee_id (for "followers of" query)       │
│                                                                  │
│  Counts (cached in Redis):                                       │
│  ├── user:{id}:follower_count → 142M                             │
│  ├── user:{id}:following_count → 847                             │
│  └── Updated asynchronously (eventually consistent)              │
│                                                                  │
│  SCALE:                                                          │
│  ├── 1B users × avg 200 following = 200B edges                   │
│  ├── Each edge: ~24 bytes → ~4.8 TB total                        │
│  ├── Sharded across ~50 database nodes                           │
│  └── Hot data (active users) cached in Redis                     │
│                                                                  │
│  CELEBRITY DETECTION:                                            │
│  └── Maintain a celebrity list (updated hourly):                 │
│      SELECT followee_id FROM follows                             │
│      GROUP BY followee_id                                        │
│      HAVING COUNT(*) > 500000                                    │
│      → Materialized view, used by fan-out service                │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Search & Trending

```
┌──────────────────────────────────────────────────────────────────┐
│                    SEARCH SERVICE                                │
│                                                                  │
│  INDEXING PIPELINE:                                              │
│  Tweet published → Kafka → Search Indexer → Elasticsearch        │
│                                                                  │
│  INDEX SCHEMA:                                                   │
│  ├── tweet_id (keyword)                                          │
│  ├── text (full-text, analyzed with language-specific tokenizer) │
│  ├── author_id, author_handle (keyword)                          │
│  ├── hashtags (keyword array)                                    │
│  ├── created_at (date)                                           │
│  ├── lang (keyword)                                              │
│  ├── engagement_score (float, for ranking)                       │
│  └── geo (geo_point, if geotagged)                               │
│                                                                  │
│  SEARCH RANKING:                                                 │
│  score = text_relevance × recency_boost × engagement_score       │
│          × author_authority                                      │
│                                                                  │
│  SCALE:                                                          │
│  ├── 500M tweets/day indexed within seconds of posting           │
│  ├── Index size: ~100 TB (searchable window = 7-30 days)         │
│  ├── Elasticsearch cluster: ~100+ nodes                          │
│  └── Older tweets archived, searchable via cold storage query    │
```

```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
│                    TRENDING TOPICS                               │
│                                                                  │
│  PIPELINE:                                                       │
│  Tweets → Extract hashtags/keywords → Sliding window counter     │
│                                                                  │
│  ALGORITHM:                                                      │
│  ├── Count hashtag frequency in 5-min tumbling windows           │
│  ├── Compare current window to historical baseline               │
│  ├── Trending = significant spike over baseline                  │
│  │   (not just high volume — spike matters more)                 │
│  ├── Filter: Remove always-popular terms (#news, #love)          │
│  └── Regional: Compute per country/city using geo data           │
│                                                                  │
│  IMPLEMENTATION:                                                 │
│  ├── Stream processor (Kafka Streams / Flink)                    │
│  ├── Count-min sketch for approximate frequency counting         │
│  ├── Redis sorted sets for top-K per region                      │
│  └── Update trending list every 1-5 minutes                      │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Notification Service

```
┌──────────────────────────────────────────────────────────────────┐
│                    NOTIFICATION SERVICE                          │
│                                                                  │
│  TRIGGERS:                                                       │
│  ├── Someone likes your tweet                                    │
│  ├── Someone retweets your tweet                                 │
│  ├── Someone replies to your tweet                               │
│  ├── Someone mentions you (@handle)                              │
│  ├── New follower                                                │
│  └── DM received                                                 │
│                                                                  │
│  PIPELINE:                                                       │
│  Event (like, retweet, ...) → Kafka → Notification Worker        │
│       │                                                          │
│       ▼                                                          │
│  ┌──────────────────────────────────────────────────┐            │
│  │  Notification Worker:                            │            │
│  │  1. Check user preferences (push enabled? muted?)│            │
│  │  2. Aggregate if burst (100 likes → "100 people  │            │
│  │     liked your tweet" not 100 separate notifs)   │            │
│  │  3. Determine delivery channel:                  │            │
│  │     ├── In-app (always, stored in notification DB)│           │
│  │     ├── Push (APNs / FCM if user has push enabled)│           │
│  │     └── Email (daily digest for inactive users)  │            │
```

```
│  │  4. Write to notification store + send push            │        │
│  └──────────────────────────────────────────────────────┘        │
│                                                                   │
│  AGGREGATION:                                                     │
│  ├── Window: 5 minutes                                            │
│  ├── "Jane and 42 others liked your tweet"                        │
│  └── Prevents notification bombing on viral tweets                │
│                                                                   │
│  CELEBRITY NOTIFICATIONS:                                         │
│  ├── If tweet gets 1M likes, DON'T send 1M push notifs           │
│  ├── Aggregate: "Your tweet reached 1M likes!"                    │
│  └── Mentions: Only notify for mentions by followed users         │
│     (to prevent @spam)                                            │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

Key Components

Component	Technology	Purpose
Tweet Store	Vitess (sharded MySQL) / CockroachDB	Persistent tweet storage, sharded by tweet_id
Timeline Cache	Redis Cluster (sorted sets)	Pre-computed timelines, O(1) reads
Social Graph	Sharded MySQL + Redis	Follow relationships, follower/following queries
Fan-out Service	Kafka + worker pool	Distribute tweets to followers' timeline caches
Search Index	Elasticsearch / Lucene	Full-text tweet search, hashtag search
Trending Engine	Kafka Streams / Flink + Count-min sketch	Real-time trending topic detection
Ranking Service	LightGBM / neural network	Algorithmic feed scoring
Media Storage	S3/Blob Storage + CDN	Image/video storage and global delivery
Notification Service	Kafka + APNs/FCM	Push notifications, in-app notifications
ID Generator	Snowflake	Globally unique, time-sortable tweet IDs

Rate Limiter	Redis (token bucket)	Per-user API rate limiting
CDN	CloudFront / Akamai	Static assets, media, JS/CSS bundles
Metrics	Prometheus + Grafana	System monitoring, SLO tracking

Scalability Considerations

1. Timeline Cache Scaling

```
Challenge: 300M users × 12.5 KB per timeline = 3.5 TB in Redis

Solutions:
├── Only cache ACTIVE users (logged in within 7 days)
│   → reduces to ~100M users → ~1.2 TB
├── Evict inactive timelines (rebuild on next login from DB)
├── Redis Cluster: 50 nodes, 64 GB each, hash-slot sharding
├── Hot/cold: Popular users cached longer, LRU eviction
└── Trim timelines to 800 entries (older = query from DB)
```

2. Fan-Out Throughput

```
Challenge: 500M tweets/day with avg 200 followers = 100B cache writes/day

Solutions:
├── Kafka partitioning: Partition tweet events by author_id
├── Worker pool: 100+ fan-out workers, each handles 1M writes/s
├── Redis pipelining: Batch 100 ZADD commands per round-trip
├── Skip celebrities: Eliminate 100M+ write spikes entirely
└── Priority queue: Fan-out high-engagement tweets first
```

3. Database Sharding

```
┌──────────────────────────────────────────────────────────┐
│                    SHARDING STRATEGY                     │
│                                                          │
│  Tweets: Shard by tweet_id (Snowflake ID, time-based)    │
│  ├── Recent tweets on fewer, faster shards               │
│  ├── Old tweets on many, cheaper shards                  │
│  └── Hot shard mitigation: Snowflake has machine_id bits │
│                                                          │
│  Social graph: Shard by follower_id                      │
│  ├── "Who do I follow?" = single-shard query             │
│  ├── "Who follows me?" = scatter-gather (cross-shard)    │
│  └── Cache follower lists in Redis to avoid scatter-gather │
```

```
│                                                                  │
│  User profiles: Shard by user_id                                 │
│  └── Simple key-value, mostly cached                             │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

4. Handling Viral Tweets

```
Problem: A viral tweet gets 1M likes/min → extreme write load on tweet_counts row

Solutions:
├── Counter sharding: Split like_count across 100 sub-counters
│   like_count = SUM(like_count_shard_0 ... like_count_shard_99)
│   Each LIKE increments a random shard → no single hot row
├── Async aggregation: Aggregate sharded counters every 5 seconds
├── Redis for real-time display: Atomic INCR in Redis, flush to DB async
└── View counts: Sample (count every 10th view, multiply by 10)
    → 10x less write load, acceptable accuracy
```

Trade-offs

Decision	Option A	Option B	Twitter's Choice
Fan-out strategy	Write (push to cache)	Read (query at read time)	**Hybrid** — push for normal users, pull for celebrities
Timeline storage	Database	In-memory cache (Redis)	**Redis** — pre-computed timelines for sub-10ms reads
Feed ordering	Chronological	Algorithmic (ranked)	**Both** — user can toggle; default is ranked
Tweet ID	UUID	Snowflake (time-sortable)	**Snowflake** — time-sortable enables cursor-based pagination
Engagement counts	Strong consistency	Eventual consistency	**Eventual** — "142 likes" vs "143 likes" doesn't matter
Search index	Index all tweets forever	Rolling window (7-30 days)	**Rolling window** — index recent tweets, archive old ones

Celebrity threshold	Fixed (500K followers)	Dynamic (based on engagement)	**Dynamic** — adjust based on fan-out capacity and tweet frequency
Media serving	Direct from storage	Via CDN	**CDN** — essential for global low-latency media delivery

---

Technologies Deep Dive

Data Layer

Technology	Purpose	Why
Vitess	Sharded MySQL for tweets + social graph	Horizontal sharding, connection pooling, used by Twitter-scale companies
Redis Cluster	Timeline cache, counters, rate limiting	Sub-millisecond reads, sorted sets are perfect for timelines
Elasticsearch	Tweet search + trending	Full-text search, real-time indexing, distributed
Kafka	Event streaming (fan-out, search indexing, notifications)	High throughput, partitioned, replay capability

Compute

Technology	Purpose	Why
Kafka Streams / Flink	Trending topics, real-time analytics	Windowed aggregations, exactly-once semantics
gRPC	Inter-service communication	Low-latency, protobuf serialization, streaming
LightGBM	Timeline ranking model	Fast inference (<20ms for 800 candidates)

Infrastructure

Technology	Purpose	Why

CDN (CloudFront/Akamai)	Media + static content delivery	Global edge caching, essential for images/video
Snowflake ID generator	Unique tweet IDs	Time-sortable, no coordination, ~4M IDs/sec/machine
Kubernetes	Service orchestration	Auto-scaling, rolling deploys, service mesh
APNs + FCM	Push notifications	iOS + Android native push channels

---

How to Explain It Well

30-Second Pitch

> "Twitter's core challenge is the fan-out problem: when a user tweets, how do you distribute it to all their followers' timelines efficiently? The solution is a hybrid approach — fan-out on write for normal users (push tweet IDs into followers' Redis timeline caches at write time) and fan-out on read for celebrities with millions of followers (query their tweets at read time and merge). Timeline reads are sub-10ms from Redis sorted sets. Tweet IDs use Snowflake for time-sortable, globally unique IDs. Search is handled by Elasticsearch with a 7-30 day rolling index window. The key numbers: 500M tweets/day, 300M DAU, 35K timeline reads/second."

Key Talking Points

1. **Fan-out problem is THE question**: Be ready to discuss push vs pull vs hybrid in depth
2. **Snowflake ID**: Time-sortable, enables cursor-based pagination, no coordination needed
3. **Redis sorted sets**: Perfect data structure for pre-computed timelines (ZADD, ZRANGE, ZREMRANGEBYRANK)
4. **Celebrity threshold**: Dynamic — too low and you lose cache benefits; too high and you get write storms
5. **Engagement counters**: Eventually consistent, sharded counters to avoid hot rows on viral tweets

Whiteboard Sequence

1. Start with tweet write → fan-out → timeline cache (the core flow)
2. Show the celebrity problem: 100M writes from 1 tweet
3. Introduce hybrid: push for normal, pull for celebrities, merge at read
4. Add search pipeline: Kafka → Elasticsearch

5. Discuss Snowflake ID and Redis sorted sets as enabling data structures

---

Common Follow-up Questions

"Why not just use fan-out on read for everyone?"

- A user follows ~200 accounts → 200 parallel DB queries per timeline read
- At 35K timeline reads/sec → 7M DB queries/sec just for timelines
- Even with caching, the merge-sort step adds latency
- Pre-computed timelines in Redis give us **< 10ms reads** vs **200-500ms** with fan-out on read
- But for celebrities, the **write cost** of fan-out on write is worse than the **read cost** of fan-out on read

"How do you handle a user with 100M followers posting 50 tweets/day?"

- **Don't fan out**: Celebrity tweets stay in the tweets DB only
- At **read time**, fetch the celebrity's recent tweets (one indexed query by author_id)
- Most users follow < 20 celebrities → 20 small DB queries merged with Redis timeline
- The queries are fast because tweets are indexed by `(author_id, created_at DESC)`
- This is the entire point of the hybrid model

"How do you handle the thundering herd when a celebrity tweets?"

- **No thundering herd** — because celebrity tweets aren't fanned out
- Followers' next timeline reads naturally pick up the tweet via fan-out on read
- The celebrity's tweet row may get hot for reads → **cache it** in a tweet object cache (Redis/Memcached)
- Celebrity tweet cache TTL: 5 minutes (short enough to stay fresh)

"How does cursor-based pagination work with Snowflake IDs?"

- Snowflake IDs are **time-ordered**: higher ID = newer tweet
- First page: `ZREVRANGE user:B:timeline 0 19` → returns 20 newest tweet IDs
- Last tweet ID on page = `cursor` for next page
- Next page: `ZREVRANGEBYSCORE user:B:timeline (cursor -inf LIMIT 0 20`
- No offset-based pagination → consistent results even as new tweets arrive

"How do you implement 'For You' recommendations?"

- Beyond the follow graph: recommend tweets from accounts the user **doesn't** follow

- **Collaborative filtering**: "Users similar to you also engaged with these tweets"
- **Content-based**: Match tweet topics to user's interest profile
- **Social signals**: "Your friends liked this tweet" (second-degree engagement)
- These are injected as candidates into the ranking pipeline alongside the follow-based timeline
- Typically 20-40% of the ranked feed is from non-followed accounts